Homework 6

Collaboration with other students are encouraged; remember to name anyone you collaborate with and to write up your final solution by yourself.

When writing proofs, mathematical clarity is just as important as correctness and rigor. Please keep this in mind and try to adhere to proof writing conventions when typing up solutions.

The first two questions are meant for additional review. We highly recommend you give these questions a try before the exam.

1. Let $G = (V, E)$ be a directed graph. We claim the removal of any non-cut edge in G increases the number of strongly-connected components. Prove this claim, or provide a counter example of no more than 4 nodes.

2. Suppose a city has 10 trolleys to cover its road network and so needs to solve the following problem: given a connected, undirected graph $G = (V, E)$ with positive edge weights, find a subgraph $A \subseteq E$ so that the graph (V, A) has at most 10 connected components and the sum of weights of edges in A is minimal given this constraint. Let $n = |V|$ and $m = |E|$. Assume $n > 10$.

   (a) How many components will (V, A) have in an optimal solution? Explain in one or two sentences.

   (b) Describe a polynomial-time algorithm to find an optimal A.

   (c) Prove that your algorithm returns an optimal A.

3. A robot starts at position 0 on a line of positions $0, 1, 2, \ldots, n$. Its goal is to reach position n. For each position $j = 1, 2, \ldots, n$, landing on position j gives a known reward r_j. At any position $i < n$, the robot may choose one of the following actions:

   - Move right by 1 position, from i to $i + 1$, with cost c_1,
   - Move right by 2 positions, from i to $i + 2$, with cost c_2, provided $i + 2 \leq n$.

   However, the robot may use the 2-step move at most K times total during the entire trip. The robot stops as soon as it reaches position n. Its objective is to maximize total reward collected minus total movement cost.

   Let $V(i, k)$ denote the maximum total net payoff the robot can still obtain starting from position i when it has exactly k uses of the 2-step move remaining.

   (a) What are the DP states in this problem? What are the possible actions from a state (i, k)?

   (b) Write a recurrence for $V(i, k)$, including the correct base cases. Your answer should clearly explain why the recurrence is correct.

   (c) Give a bottom-up dynamic programming algorithm that computes $V(0, K)$. Your algorithm should also store enough information to recover an optimal sequence of moves.

   (d) What is the running time of your algorithm as a function of n and K? Briefly justify your answer.

   (e) Consider the following greedy rule: "At each step, choose the feasible move with the larger immediate net gain." Here "feasible" means the move is allowed by the remaining number of 2-step moves. Give a small example showing that this rule does not always produce an optimal solution.